Dev2 보안스캔 — A커머스(e-commerce) 전용 커스텀 검증 규칙

> **대상 에이전트**: `security_scan` (Dev2) **적용 범위**: PR diff에 포함된 변경 파일 (전 언어, `.py` 중심) **표준 도구와의 관계**: bandit(`SEC-B*`) / semgrep(`OWASP-*`) / pip_audit / npm_audit 가 잡지 못하는 **커머스 보안 정책 (PCI-DSS·결제·개인정보)** 위반을 보완한다. **구현**: `dev2/tools/common/custom_security_tool.py` (`run_custom_security`, AST+패턴)

---

1. 왜 커스텀 규칙이 필요한가

bandit·semgrep은 "범용 CWE" 패턴(SQLi, XSS, 약한 해시 등)을 본다. 하지만 A커머스는 **카드정보(PAN/CVV) 취급, 결제 PG Webhook 서명, 결제 시크릿 패턴, 주문 소유권 검사** 같은 도메인 정책이 있고, 이를 어겨도 표준 스캐너는 통과시킨다.

표준 스캐너가 보는 것	A커머스 커스텀 규칙이 보는 것
범용 SQLi/XSS/CWE 패턴	결제 Webhook 서명 검증 누락
일반 하드코딩 비밀번호	결제 키 패턴(`sk_live_`, `imp_`, `live_`)
일반 약한 해시 호출	"비밀번호" 맥락의 md5/sha1
—	**카드번호/CVV 저장·로깅(PCI-DSS)**, 타인 주문 접근(IDOR)

---

2. 규칙 ID 체계

```
ASEC-NNN  = A社 Security 커스텀 규칙
AAUTH-NNN = A社 인증/인가 커스텀 규칙
```

- 평가 데이터셋(`pj3_data`)의 `SEC-001~006`, `AUTH-001` 과 **개념 매핑**되며, 커스텀 탐지 결과는 `Issue.concept_id` 에 매핑 룰(SEC-001 등)을 기록해 M4 학습 연계.
- 표준 도구 유래 ID(`SEC-B608`, `OWASP-sql-injection`)와 네임스페이스가 겹치지 않는다.

severity 매핑 기준 (bandit HIGH/MEDIUM/LOW 매핑과 일관)

severity	의미	HITL 처리(CLAUDE.md §6)
Critical	즉시 침해·규제 위반 가능	Critical ≥1 → Slack 즉시 에스컬레이션
Major	조건부 침해·정책 위반	Major ≥3 → rework
Minor	위생·권고	코멘트 반영

> `issue_type` 은 모두 `"Vulnerability"` 로 기록.

---

3. 커스텀 규칙 목록

ASEC-001 — 동적 문자열 SQL 조합 (파라미터 바인딩 강제)

- **severity**: Critical · 매핑: `SEC-001`
- **탐지 패턴**: `SELECT/INSERT/UPDATE/DELETE` 문자열이 f-string·`%`·`.format()`·`+` 로 외부 입력(주문ID·검색어)과 결합. (bandit B608 보완)
- **suggestion**: "파라미터 바인딩(플레이스홀더)을 사용하세요."

ASEC-002 — 결제 PG Webhook 서명 검증 누락

- **severity**: Critical
- **근거**: 서명 검증 없는 Webhook은 **위조 결제완료 통보**로 주문을 임의 확정시킬 수 있다.
- **탐지 패턴**: 이름에 `webhook` 이 포함된 핸들러 본문에 `compare_digest`/`verify_signature` 호출이 없음. (Stripe/토스/아임포트 등 PG 서명)
- **suggestion**: "PG 서명을 `hmac` 으로 계산해 `compare_digest` 로 헤더와 비교한 뒤 본문을 처리하세요."

> *ASEC-003(서명 헤더 부재 시 거부)은 ASEC-002 의 서명 검증 흐름에 포함해 함께 처리한다.*

ASEC-004 — 카드정보(PAN/CVV) 저장·로깅 (PCI-DSS)

- **severity**: Critical
- **근거**: PCI-DSS — **PAN은 토큰화/마스킹, CVV는 어떤 경우에도 저장 금지**.
- **탐지 패턴**: `card_number`/`pan`/`cvv`/`cvc` 등 변수명이 `logger`/`print` 인자 또는 `execute`/`save`/`insert` 등 저장 호출에 포함.
- **suggestion**: "PAN/CVV는 저장·로깅 금지. PG 토큰화(빌링키)를 사용하고 CVV는 보관하지 마세요."

ASEC-005 — 결제/시크릿 키 하드코딩

- **severity**: Critical · 매핑: `SEC-006`
- **탐지 패턴**: 소스에 결제·클라우드·DB 시크릿 리터럴 — `sk_live_`/`pk_live_`(Stripe), `imp_`(아임포트), `live_…`(토스/PG), `AKIA…`(AWS), `postgresql://user:pass@`, `JWT_SECRET/API_KEY = "..."`. (bandit B105/B106 보완)
- **suggestion**: "`os.environ[...]`/시크릿 매니저로 이동하세요. `.env` 커밋 금지."

ASEC-006 — 비밀번호 약한 해시

- **severity**: Major · 매핑: `SEC-005`
- **탐지 패턴**: "password/passwd/pw" 맥락에서 `hashlib.md5`/`sha1` 사용.
- **suggestion**: "bcrypt/argon2/scrypt를 사용하세요."

ASEC-007 — 경로 탐색 (영수증/송장 다운로드)

- **severity**: Major · 매핑: `SEC-003`
- **탐지 패턴**: 사용자 입력 `filename`/`receipt`/`invoice` 를 `BASE/ name` 으로 결합 후 `open`/`read_text`. `resolve()` + base 접두 검증 부재.
- **suggestion**: "`(BASE / name).resolve()` 후 base 경로로 시작하는지 검증하세요."

ASEC-008 — Shell 명령 인젝션

- **severity**: Critical · 매핑: `SEC-004`
- **탐지 패턴**: `os.system(f"...{var}")`, `subprocess.*(..., shell=True)`.
- **suggestion**: "`subprocess.run([...], shell=False)` 인자 리스트로 호출하세요."

AAUTH-001 — 인가(권한) 검사 누락 (타인 주문 접근, IDOR)

- **severity**: Major · 매핑: `AUTH-001`
- **탐지 패턴**: `cancel_*`/`delete_*`/`update_*`/`refund_*`/`get_order*`/`get_payment*` 함수에 `current_user`/`request.user`/`owner`/`authorize` 등 소유자·권한 검사가 본문에 없음.
- **suggestion**: "변경/조회 전 `current_user` 와 리소스 소유자 일치 또는 관리자 권한을 확인하세요."

ASEC-009 — 개인정보 민감정보 로깅

- **severity**: Minor
- **탐지 패턴**: `logger`/`print` 인자에 `password`·`token`·`secret`·`Authorization`·주민번호 등 포함.
- **suggestion**: "민감 값은 마스킹하거나 로그에서 제외하세요."

---

4. 도구 연동 방법

1. `dev2/tools/common/custom_security_tool.py` — `run_custom_security(files) -> list[dict]` (AST + 패턴 스캔). `rule_id`는 `ASEC-NNN`/`AAUTH-NNN`, `concept_id`에 매핑 룰(SEC-001 등) 기록. 미설치/파싱 실패 시 **빈 리스트 반환** (CLAUDE.md §12). (하드코딩 시크릿 텍스트 스캔은 파싱 실패와 무관하게 동작)
2. `tools/router.py` 보안 도구 목록에 `"custom_security"` 추가.
3. `nodes/run_tools.py _run_python()` 에서 `security_parts` 로 합류.
4. 결과는 `security_raw` → `enriched`(pgVector 유사 이슈 + LLM 판정) → `final_output` → `AgentOutput(agent="security_scan")`.
5. **Critical ≥ 1** 이면 Dev4 3-Way 판정에서 `escalate` → Slack 알림 (CLAUDE.md §4, §6).

> **계약 불변식**: 모든 커스텀 보안 이슈는 `schema.Issue` 9개 필수 필드 + `severity` enum을 지킨다.

---

5. 요약 표

ID	규칙	severity	평가셋 매핑
ASEC-001	동적 문자열 SQL 조합	Critical	SEC-001
ASEC-002	결제 Webhook 서명 검증 누락	Critical	—
ASEC-004	카드정보(PAN/CVV) 저장·로깅	Critical	— (PCI-DSS)
ASEC-005	결제/시크릿 키 하드코딩	Critical	SEC-006
ASEC-006	비밀번호 약한 해시	Major	SEC-005
ASEC-007	경로 탐색(영수증/송장)	Major	SEC-003
ASEC-008	Shell 명령 인젝션	Critical	SEC-004
AAUTH-001	인가 검사 누락(IDOR)	Major	AUTH-001

ID	규칙	severity	평가셋 매핑
ASEC-009	개인정보 민감정보 로깅	Minor	—